

Linux 进程间通信

一、 进程互斥所涉及的系统调用

`lockf(files,function,size)`

作用：对指定文件的指定区域（由 `size` 指示）进行加锁或解锁，以实现进程的同步与互斥。

本函数的头文件为

```
#include <unistd.h>
```

参数定义：

```
int lockf(files,function,size)
```

```
int files,function;
```

```
long size;
```

其中：`files` 是文件描述符；`function` 是锁定和解锁：1 表示锁定，0 表示解锁。`size` 是锁定或解锁的字节数，为 0，表示从文件的当前位置到文件尾。

二、Linux 进程间通信

进程间通信就是在不同进程之间传播或交换信息，进程间通信的目的在于实现数据传输、数据共享、事件通知、资源共享和进程控制等。Linux 进程间通信（IPC）主要由早期 Unix 进程间通信、基于 System V 进程间通信、基于 Socket 进程间通信和 POSIX 进程间通信几部分发展而来。Unix 进程间通信的方式包括管道、FIFO、信号；System V 进程间通信方式包括 System V 消息队列、System V 信号灯、System V 共享主存等，通信进程局限在单个计算机内；基于 Socket 进程间通信则越过了该界限，能实现多机间进程的通信；POSIX 进程间通信包括 POSIX 消息队列、POSIX 信号灯、POSIX 共享主存。

现在 Linux 使用的进程间通信方式主要有以下几种。

1. 管道（pipe）和有名管道（named pipe）

在 Linux 系统中，管道是一种很重要的通信方式，它将一个程序的输出直接连接到一个程序的输入。从本质上说，管道也是一种文件，但它又和一般的文件有所不同，管道具有以下特点：

- 管道是半双工的，数据只能单向流动；需要双方通信时，需要建立起两个管道。
- 管道只能用于父子进程或者兄弟进程之间（具有亲缘关系的进程）。
- 管道对于管道两端的进程而言，就是一个文件，但它不是普通的文件，不属于某种文件系统，而是单独构成一种文件系统，并且只存在于主存中。
- 进程向管道中写的内容被管道另一端的进程读出。写入的内容每次都添加在管道缓冲区的末尾，并且每次都是从缓冲区的头部读出数据。

注意：从管道读数据是一次性操作，数据一旦被读，它就从管道中被抛弃，释放空间以便写更多的数据。

2. 信号（signal）

信号是比较复杂的通信方式，类似于 Windows 下的消息，用于通知接受进程有某种事件发生。除了用于进程间通信外，进程还可以发送信号给进程本身。Linux 除了支持 Unix 早期信号语义函数 `signal` 外，还支持语义符合 Posix.1 标准的信号函数 `sigaction`。

信号机制是 Unix 系统中最为古老的进程之间的通信机制，用于在一个或多个进程间传递异步信号。信号是对中断机制的一种模拟，故在早期的 Unix 版本中又把它称为软中断。很多条件可以产生一个信号，如用户按某些终端键、无效的存储访问、管道的读进程终止后有进程写此管道、进程所设置的时间超时等。

3. 消息（message）队列

消息队列也叫报文队列，是消息的链接表。目前主要有两种类型的消息队列：POSIX 消息队列和系统 V 消息队列，系统 V 消息队列目前被大量使用。考虑到程序的可移植性，新开发的应用程序应尽量使用 POSIX 消息队列。

消息队列用于运行同一台机器上的进程间通信，和管道很相似，有足够权限的进程可以向消息队列中发送某种类型的消息，被赋予读权限的进程可以从消息队列中读取某种类型的消息。但消息主存可以根据需要自行定义，从而使消息队列克服了信号承载信息量少、管道只能承载无格式字节流以及缓冲区大小受限等缺点，在实际编程中应用较广。可以用流管道或套接口的方式取代它。

4. 共享主存

共享主存是在系统内核分配的一块缓冲区，多个进程都可以访问该缓冲区。采用共享主存通信的一个显而易见的好处是效率高，因为进程可以直接读写主存，而不需要复制任何数据，避免了在内核空间与用户空间的切换。对于像管道和消息队列等通信方式，则需要在内核空间和用户空间进行四次数据复制，而共享主存只需要复制两次数据：从输入文件到共享主存区和从共享主存区到输出文件。实际上，进程之间在共享主存时，并不总是读写少量数据后就解除映射，有新的通信时，再重新建立共享主存区域；而是保持共享区域，直到通信完毕为止，这样，数据内容一直保存在共享主存中，并没有写回文件。共享主存中的内容往往是在解除映射时才写回文件的。因此，采用共享主存的通信方式效率是非常高的。

共享主存类似于 Windows 环境编程中的主存影像文件，在 Linux 系统中针对其它通信机制运行效率较低而设计，多用于存储应用程序的配置信息。这个机制的不利方面是其同步和协议都不受程序员控制，必须确保将句柄传递给子进程和线程，因此往往与其它通信机制，如信号量结合使用。

5. 信号量

信号量及信号量上的操作是一种解决同步、互斥问题的较通用的方法，并在很多操作系统中得以实现，Linux 改进并实现了这种机制。信号量也称为信号灯，用来协调不同进程间的数据对象，最主要的应用就是共享主存方式的进程间通信。本质上，信号量是一个计数器，用来记录对某个资源（如共享主存）的存取

状况。

维护信号量状态的是 Linux 内核操作系统而不是用户进程。信号量主要提供对进程间共享资源访问控制的手段，用来保护共享资源。除了用于访问控制外，还可以用于进程间及同一进程不同线程间的进程同步。

6. 套接口 (socket)

套接口也称为套接字，可用于不同机器之间的进程通信。一个套接口可以看作进程间通信的端点 (endpoint)，每个套接口的名字都是唯一的，其它进程可以发现、连接并且与之通信。通信域用来说明套接口通信用的协议，不同的通信域有不同的通信协议以及套接口的地址结构等，因此，创建一个套接口时，要指明它的通信域。比较常见的是 Unix 域套接口（采用套接口机制实现单机内的进程间通信）及网际通信域。

套接口起初由 Unix 系统的 BSD 分支开发，由于其功能强大、接口友好，现在已经移植到大多数操作系统中，成为事实上的网络编程接口。套接口是网络编程中一个非常重要的概念，Linux 以文件的形式实现套接口，与套接口相应的文件属于 sockfs 特殊文件系统，创建一个套接口就是在 sockfs 中创建一个特殊文件，并建立起为实现套接口功能的相关数据结构。换句话说，对每一个新创建的 BSD 套接口，Linux 内核都将在 sockfs 特殊文件系统中创建一个新的 inode。

三、信号

1、信号的基本概念

每个信号都对应一个正整数常量(称为 signal number,即信号编号。定义在系统头文件<signal.h>中)，代表同一用户的诸进程之间传送事先约定的信息的类型，用于通知某进程发生了某异常事件。每个进程在运行时，都要通过信号机制来检查是否有信号到达。若有，便中断正在执行的程序，转向与该信号相对应的处理程序，以完成对该事件的处理；处理结束后再返回到原来的断点继续执行。实质上，信号机制是对中断机制的一种模拟，故在早期的 UNIX 版本中又把它称为软中断。

信号与中断的相似点：

- (1) 采用了相同的异步通信方式;
- (2) 当检测出有信号或中断请求时, 都暂停正在执行的程序而转去执行相应的处理程序;
- (3) 都在处理完毕后返回到原来的断点;
- (4) 对信号或中断都可进行屏蔽。

信号与中断的区别:

- (1) 中断有优先级, 而信号没有优先级, 所有的信号都是平等的;
- (2) 信号处理程序是在用户态下运行的, 而中断处理程序是在核心态下运行;
- (3) 中断响应是及时的, 而信号响应通常都有较大的时间延迟。

信号机制具有以下三方面的功能:

- (1) 发送信号。发送信号的程序用系统调用 `kill()` 实现;
- (2) 预置对信号的处理方式。接收信号的程序用 `signal()` 来实现对处理方式的预置;
- (3) 收受信号的进程按事先的规定完成对相应事件的处理。

2、信号的发送

信号的发送, 是指由发送进程把信号送到指定进程的信号域的某一位上。如果目标进程正在一个可被中断的优先级上睡眠, 核心便将它唤醒, 发送进程就此结束。一个进程可能在其信号域中有多个位被置位, 代表有多种类型的信号到达, 但对于一类信号, 进程却只能记住其中的某一个。

进程用 `kill()` 向一个进程或一组进程发送一个信号。

3、对信号的处理

当一个进程要进入或退出一个低优先级睡眠状态时, 或一个进程即将从核心态返回用户态时, 核心都要检查该进程是否已收到软中断。当进程处于核心态时, 即使收到软中断也不予理睬; 只有当它返回到用户态后, 才处理软中断信号。对软中断信号的处理分三种情况进行:

- (1) 如果进程收到的软中断是一个已决定要忽略的信号 (`function=1`), 进程不做任何处理便立即返回;
- (2) 进程收到软中断后便退出 (`function=0`);

(3) 执行用户设置的软中断处理程序。

四、信号通信函数

1、kill()

系统调用格式

```
int kill(pid,sig)
```

参数定义

```
int pid,sig;
```

其中，pid 是一个或一组进程的标识符，参数 sig 是要发送的软中断信号。

(1) pid>0 时，核心将信号发送给进程 pid。

(2) pid=0 时，核心将信号发送给与发送进程同组的所有进程。

(3) pid=-1 时，核心将信号发送给所有用户标识符真正等于发送进程的有效用户标识号的进程。

2、signal()

预置对信号的处理方式，允许调用进程控制软中断信号。

系统调用格式

```
signal(sig,function)
```

头文件为

```
#include <signal.h>
```

参数定义

```
signal(sig,function)
```

```
int sig;
```

```
void (*func) ()
```

其中 sig 用于指定信号的类型，sig 为 0 则表示没有收到任何信号，余者如下表：

值	名 字	说 明
01	SIGHUP	挂起 (hangup)
02	SIGINT	中断，当用户从键盘按 ^c 键或 ^break 键时
03	SIGQUIT	退出，当用户从键盘按 quit 键时
04	SIGILL	非法指令
05	SIGTRAP	跟踪陷阱 (trace trap)，启动进程，跟踪代码的执行
06	SIGIOT	IOT 指令
07	SIGEMT	EMT 指令

08	SIGFPE	浮点运算溢出
09	SIGKILL	杀死、终止进程
10	SIGBUS	总线错误
11	SIGSEGV	段违例 (segmentation violation)，进程试图去访问其虚地址空间以外的位置
12	SIGSYS	系统调用中参数错，如系统调用号非法
13	SIGPIPE	向某个非读管道中写入数据
14	SIGALRM	闹钟。当某进程希望在某时间后接收信号时发此信号
15	SIGTERM	软件终止 (software termination)
16	SIGUSR1	用户自定义信号 1
17	SIGUSR2	用户自定义信号 2
18	SIGCLD	某个子进程死
19	SIGPWR	电源故障

function: 在该进程中的一个函数地址，在核心返回用户态时，它以软中断信号的序号作为参数调用该函数，对除了信号 SIGKILL，SIGTRAP 和 SIGPWR 以外的信号，核心自动地重新设置软中断信号处理程序的值为 SIG_DFL，一个进程不能捕获 SIGKILL 信号。

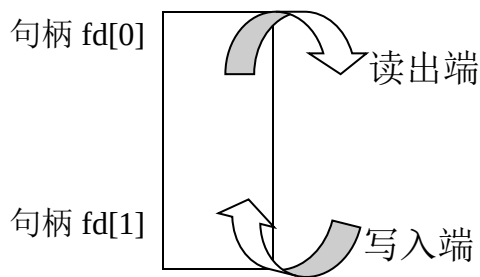
function 的解释如下：

- (1) function=1 时，进程对 sig 类信号不予理睬，亦即屏蔽了该类信号；
- (2) function=0 时，缺省值，进程在收到 sig 信号后应终止自己；
- (3) function 为非 0，非 1 类整数时，function 的值即作为信号处理程序的指针。

五、管道

1. 什么是管道

管道是一种遵循先进先出原则的数据结构，先进入管道的数据，也能先从管道中读出。所谓管道，是指能够连接一个写进程和一个读进程的、并允许它们以生产者—消费者方式进行通信的一个共享文件，又称为 pipe 文件。由写进程从管道的写入端（句柄 1）将数据写入管道，而读进程则从管道的读出端（句柄 0）读出数据。数据一旦读取后，就会在管道中自动删除。



管道通信是一种常用的单向进程间通信机制，它以管道数据结构作为内部数据存储方式，以文件系统作为数据存储媒体，将一个写进程和一个读进程连接在一起，实现两个进程间的通信。管道作为一临界资源，使用过程中父子进程之间除了需要写同步以外，在对管道进行读写操作时还需要互斥进入。

2. 管道的类型

Linux 系统中有两种管道，分别是无名管道和有名管道，在这里只介绍无名管道。管道中的数据格式是字符流。

无名管道为创建管道的进程及其子孙提供传递消息的信道，逻辑上它是由操作系统在主存中创建的临时文件实现的，被看作管道文件，物理上则由文件系统的高速缓冲区构成。由于涉及文件系统，无名管道文件描述符只能供创建管道的进程及其子孙进程共享使用，因为子进程能够继承父进程打开的所有文件，所以能够继承父进程所创建的无名管道文件。因此，进程间使用管道的通信方式仅限于同一家族的进程之间使用。

创建一个无名管道的函数是 `int pipe(int fd[2])`，它同时分配两个文件描述符，一个用于读，另一个用于写，它们是管道的两端。从一端写入的内容，可以从另一端读出。显然，这对于同一个进程来说，没有任何用处。但在创建新进程时，子进程继承父进程的所有文件描述符，子进程同时也拥有 `pipe()` 所创建的两个文件描述符。因此，如果父进程写入管道的一端，子进程可以从管道的另一端读出。反之亦然，这样就实现了父子进程、兄弟进程之间的通信。

管道两端可分别用描述符 `fd[0]` 和 `fd[1]` 描述。`fd[0]` 只能用于读，称为管道读端；`fd[1]` 只能用于写，称为管道写端。一般文件的 I/O 函数都可用于管道，如 `close()`、`read()`、`write()` 等。

从管道中读取数据的规则为：

- 如果管道的写端不存在，则认为已经读到了数据的末尾，读函数返回的读出字节数为 0。

- 当管道的写端存在时，如果请求的字节数大于 PIPE_BUF，则返回管道中现有的数据字节数；如果请求的字节数目不大于 PIPE_BUF，若管道中数据量小于请求的数据量，则返回管道中现有的数据字节数，否则返回请求的字节数（管道中数据量不小于请求的数据量）。

向管道中写入数据的规则为：

向管道中写入数据时，Linux 将不保证写入数据的原子性，管道缓冲区一有空闲区域，写进程就会试图向管道写入数据。如果读进程不读走管道缓冲区中的数据，那么写操作将一直阻塞。要注意，只有在管道的读端存在时，向管道中写入数据才有意义。否则，向管道中写入数据的进程将收到内核传来的 SIGPIPE 信号，应用程序可以处理该信号，也可以忽略该信号（默认动作是应用程序终止）。

六、管道通信函数

1.pipe()

作用：

建立一无名管道。调用成功时，返回 0；错误时返回-1。

系统调用格式：

pipe(filedes)

参数定义

```
int pipe(filedes);
```

```
int filedes[2];
```

其中，filedes[1]是写入端，filedes[0]是读出端。

该函数使用头文件如下：

```
#include <unistd.h>
```

```
#include <signal.h>
```

```
#include <stdio.h>
```

```
2.read( )
```

系统调用格式:

```
read(fd,buf,nbyte)
```

功能: 从 fd 所指示的文件中读出 nbyte 个字节的数据, 并将它们送至由指针 buf 所指示的缓冲区中。如该文件被加锁, 等待, 直到锁打开为止。

参数定义:

```
int read(fd,buf,nbyte);
```

```
int fd;
```

```
char *buf;
```

```
unsigned nbyte;
```

```
3.write( )
```

系统调用格式

```
write(fd,buf,nbyte)
```

功能: 把 nbyte 个字节的数据, 从 buf 所指向的缓冲区写到由 fd 所指向的文件中。如文件加锁, 暂停写入, 直至开锁。

参数定义同 read()。

注意: 管道为一临界资源, 使用过程中父子进程之间除了需要读写同步以外, 在对管道进行读写操作时还需要互斥进入, 可以使用对文件上锁和开锁的系统调用 lockf(files,function,size)。